CLAUDE.md / AGENTS.md - Cheatsheet

One page. Print landscape. Pin near monitor.

Which file for what?

	CLAUDE.md	AGENTS.md
Who reads it directly	Claude Code (every session, into context window)	Codex directly; Claude Code only via @AGENTS.md import inside CLAUDE.md (or symlink). Claude Code does NOT read AGENTS.md natively.
Purpose	Conventions, forbidden patterns, testing rules, project memory	Cross-tool orchestration protocol: roles, handoffs, test authority, multi-agent contracts
Location	Project root (or .claude/CLAUDE.md)	Project root, alongside CLAUDE.md
Bridging the gap	n/a	Reference it from CLAUDE.md: See @AGENTS.md for multi-agent orchestration rules. Or symlink it.
Length budget	?200 lines	?200 lines

Loading / import model

Per the official Memory docs (https://code.claude.com/docs/en/memory), Claude Code reads CLAUDE.md, not AGENTS.md. Per the official Codex docs (https://developers.openai.com/codex/guides/agents-md), Codex reads AGENTS.md before work. These files are loaded into context as instructions; they are not hard security controls.

Common locations:

- Managed policy, e.g. /Library/Application Support/ClaudeCode/CLAUDE.md, /etc/claude-code/CLAUDE.md, or C:\Program Files\ClaudeCode\CLAUDE.md
- User rules: ~/.claude/CLAUDE.md
- Project rules: ./CLAUDE.md or ./.claude/CLAUDE.md
- Local checkout rules: ./CLAUDE.local.md (gitignored)
- Imports: @AGENTS.md, @.claude/rules/<topic>.md

Important nuance: Claude Code concatenates relevant memory files into context. If two rules contradict each other, Claude may choose one arbitrarily. The fix is not "rely on override order"; the fix is to reconcile or delete the conflict.

For AGENTS.md to be active in Claude Code: add @AGENTS.md inside your project CLAUDE.md, OR symlink AGENTS.md -> CLAUDE.md.

For GitHub Copilot: official repository instructions live in .github/copilot-instructions.md. Keep AGENTS.md as the cross-agent source and mirror only the essential rules if Copilot is part of the toolchain.

For Codex overrides: Codex checks each directory on the path to the working directory and includes at most one instruction file per directory. In each directory it prefers AGENTS.override.md, then AGENTS.md, then configured fallback filenames. Put specialized overrides close to the service or package they govern.

When the project grows, split overflow into .claude/rules/<topic>.md and import via @.claude/rules/<topic>.md in CLAUDE.md - never let any single file exceed 200 lines.

For large codebases, start Claude Code from the service/package directory you are changing. Claude walks upward and loads the relevant parent CLAUDE.md files, so root context is still available without forcing every unrelated package rule into the task. If a parent file creates contradictions, exclude irrelevant memory with claudeMdExcludes rather than adding counter-rules.

Which extension when?

```
Need | Use | Example
--- | --- | ---
Project context and recurring instructions | CLAUDE.md / .claude/rules/ | "Use pnpm, run pytest, follow PEP8."
Deterministic lifecycle automation | Hooks / hookify | Format after edits, block commit without tests, notify on completion.
External state or systems | MCP | Query Jira, fetch GitHub PRs, read Confluence, access approved databases.
Focused delegated investigation | Subagent | Research migration risks and return a summary.
Independent concurrent workstreams | Worktrees / agent teams | One agent fixes API, another updates UI, each in an isolated checkout.
```

Rule of thumb: guidance goes in CLAUDE.md; enforcement goes in hooks; external access goes through MCP; parallel work needs isolated contexts.

CLAUDE.md - minimal skeleton

```
## Project
[Stack, version, prod/non-prod, data sensitivity]

## Architecture (5 lines max)
[Controller -> Service -> Repository; package layout]

## Forbidden Patterns
- Never expose PII in logs (mask IBAN, card, account)
- Always parameterized queries (no string concat in SQL)
- Constructor injection only (no @Autowired on fields)
- No business logic in controllers

## Mistake Journal (append-only)
- [Date] Claude did X, should have done Y. Cause: missing context about Z.

## Testing
- Framework: JUnit 5 + Mockito + Testcontainers
- Run: ./gradlew test
- Every public service method has a test
```

AGENTS.md - minimal skeleton

```
## Roles
- Lead: planning + orchestration; never implements
- Implementer (subagent): one per task, fresh context, no inheritance
- Reviewer (subagent): spec compliance + code quality (two separate agents)
- QA fleet: pr-review-toolkit before human review

## Handoff Protocol
- Lead -> Implementer: task packet (Goal + Files + Forbidden + Success Criteria)
- Implementer -> Reviewer: diff + tests + claim of done
- Reviewer -> Lead: severity-ranked findings (file:line), NOT prose
- QA -> Human: GO / NO-GO with reproduction evidence

## Test Authority
- Implementer MUST run tests before claiming done
- Reviewer MUST run INDEPENDENT verification (not implementer's tests)
- Final QA: full regression + 3 random completed features

## Cross-Tool Notes
- Tool-specific config in .claude/, .cursor/, .aider/ - never collide
- AGENTS.md is the lowest-common-denominator contract
```

Common mistakes

```
Mistake | Fix
--- | ---
CLAUDE.md > 200 lines | Split into .claude/rules/<topic>.md
AGENTS.md contradicts CLAUDE.md | CLAUDE.md wins; reconcile or delete the conflict
Personal preferences in committed CLAUDE.md | Move to ~/.claude/CLAUDE.md
No Mistake Journal entries after 30 days | Either the team isn't actually using AI, OR isn't reviewing
critically - investigate
"Always use X" with no example | Add a 3-line snippet showing the right way
```

Banking-specific additions (drop into Forbidden Patterns)

- No float/double for money - BigDecimal only
- Idempotency keys required on transfer/payment endpoints
- Audit-log writes are non-cancellable (separate transaction or outbox)
- No PII in logs without masking (IBAN, card, account, name, email)
- Rate-limit on transaction endpoints (Bucket4j or Spring Cloud Gateway)

Sandbox baseline - split by scope

The settings below come in two halves. Project-level keys go into .claude/settings.json at your repo root and can be edited by any developer on the project. Managed-only keys go into the org's managed-settings file (admin-deployed; e.g. /Library/Application Support/ClaudeCode/managed-settings.json) and CANNOT be overridden from the project. Don't mix them - managed-only keys placed in project settings will be silently ignored.

Half A - Project-level (in .claude/settings.json)

```
{
  "permissions": {
    "defaultMode": "auto",
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push origin main)",
      "Bash(DROP TABLE)",
      "Bash(TRUNCATE)",
      "Read(./.env*)",
      "Write(/.env*)",
      "Write(.github/**)"
    ],
    "ask": [
      "Bash(npm install *)",
      "Bash(pip install *)",
      "WebFetch(*)"
    ]
  },
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "allowUnsandboxedCommands": false,
    "filesystem": {
      "denyRead": ["~/.aws/", "~/.ssh/", "./.env", "./secrets/*"],
      "denyWrite": ["~/", "/etc/", "/usr/**"]
    },
    "network": {
      "allowedDomains": [
        "github.com",
        "*.npmjs.org",
        "api.anthropic.com",
        "*.atlassian.net"
      ]
    }
  }
}
```

Half B - Policy floor (prefer the org's managed-settings file)

```
{
  "permissions": {
    "disableBypassPermissionsMode": "disable"
  },
  "allowManagedPermissionRulesOnly": true
}
```

Why these settings:
- permissions.defaultMode: "auto" - the recommended default. Claude's classifier decides per-action what needs approval. Far less friction than approving every file read; far smarter than blanket "yes to everything." Allow/Ask/Deny rules below are granular overrides on top of auto, not a replacement for it. (Note: the key lives inside permissions, not at the top level - easy to get wrong by hand.)
- permissions.deny - the non-negotiable list. Even if auto thinks an action is safe, deny rules win. Keep this list focused on operations that are dangerous regardless of context. Important caveat: permission rules have had documented bypass classes - Adversa AI Red Team disclosed a parser-cap bypass in April 2026 (patched in v2.1.90); eve.gd documented arbitrary-subprocess and chained-shell bypass classes the same month. Treat the deny list as defense-in-depth. The

hard boundary is the sandbox below. The Bash(DROP TABLE) pattern is illustrative - real protection uses a PreToolUse hook with case-insensitive regex, since the glob won't catch drop table lowercase or DROP\nTABLE.
- permissions.ask - the always-stop list. Auto might let these through; you want explicit human approval. Use for compliance, audit, or high-blast-radius operations.
- sandbox.enabled: true + failIfUnavailable: true - if the OS can't sandbox (e.g. unsupported platform), Claude refuses to run rather than degrading silently.
- sandbox.allowUnsandboxedCommands: false - blocks any command that the sandbox can't enforce against (e.g. binaries that escape the OS-level isolation). The default is permissive on some platforms; banking-grade deployments should explicitly set this false.
- denyRead covers .env, AWS / SSH credential dirs, and ./secrets/* at the OS level. Unlike permission rules, this can't be bypassed by a subprocess opening the file directly.
- denyWrite keeps the agent out of the home directory (outside the project), /etc (system config), and /usr (system binaries). The sandbox already denies write to the filesystem root by default, so listing / is redundant; we list ~/** explicitly because the home directory is otherwise a common write target.
- allowedDomains is the minimum for a dev session - add more only by review. Caveat: the built-in network proxy doesn't terminate TLS, so a broad allow like github.com permits domain fronting. For banking-grade isolation, deploy a TLS-terminating proxy and pin a CA cert via httpProxyPort / socksProxyPort.
- permissions.disableBypassPermissionsMode: "disable" blocks --dangerously-skip-permissions from the command line. Anthropic docs say it works from any settings scope, but for organizational policy it belongs in managed settings so individual developers cannot remove it casually.
- Managed-only allowManagedPermissionRulesOnly: true blocks per-project overrides of the managed permission set entirely. Combine with the disable above to get a real org-policy floor.

Sizing rule: if your deny + ask + allow rules together exceed ~15 entries per repo, your CLAUDE.md is probably too thin. Better-described project context lets auto make the right call without needing explicit overrides.

Caveat (TLS): the built-in network proxy does not terminate TLS. A broad allow like github.com permits domain fronting. For banking-grade isolation, deploy a TLS-terminating proxy and pin a CA cert inside the sandbox via httpProxyPort / socksProxyPort.

ITSS Workshop - 28 May 2026. Reference for the 90-day rollout.